INTERNSHIP REPORT

JYOTSANA S

JULY 2025

Contents

1 Acknowledgement **3**

2 Introduction **4**

3 Background and Research **5**
3.1 Conversational AI in Chatbots . . . 5
3.1.1 Intent Classification . . . 7
3.2 Generative AI in Chat bots . . . 11
3.3 Real Life Implementations of AI Agents . . . 15
3.3.1 AI in Health-Care . . . 15
3.3.2 AI in Education . . . 17
3.4 Natural Language Processing . . . 18
3.4.1 Natural Language Understanding . . . 20
3.5 Information Retrieval . . . 22
3.5.1 Sparse Retrieval . . . 24
3.5.2 Dense Retrieval . . . 25
3.5.3 Hybrid Retrieval . . . 26
3.6 Retrieval-Augmented Generation (RAG) . . . 27
3.7 Practical LLM Considerations . . . 30

4 Demo Report **32**
4.1 System Setup . . . 32
4.1.1 Environment Setup . . . 32
4.1.2 Docker Installation . . . 32
4.1.3 ElasticSearch Setup via Docker . . . 32
4.2 Data Preparation . . . 32
4.2.1 Dataset Collection . . . 32
4.2.2 Chunking Text Data . . . 33
4.2.3 Indexing into ElasticSearch . . . 33
4.3 Implementation and Experiments . . . 34
4.3.1 Sparse Search Queries . . . 34
4.3.2 Dense Search and Vector Indexing . . . 36
4.4 Performance and Observations . . . 37

5 Conclusion **38**

6 References **40**

1 Acknowledgement

I would like to thank the Indian Institute of Technology, Professor N S Narayanaswamy, Mr. Jaimandeep Singh and Ms. Anusha for this opportunity and for their guidance and support during this internship.

2 Introduction

Artificial intelligence is reshaping the way humans interact with machines, and nowhere is this more visible than in the domain of conversational systems. From early rule-based chatbots to today's large language models capable of nuanced, context-aware dialogue, the pace of progress has been remarkable. As these systems become embedded in healthcare, education, customer service, and enterprise workflows, understanding the technologies that power them has become increasingly important for anyone working in the field of computer science and AI.

This report documents the research and practical work undertaken during an internship at the Indian Institute of Technology, conducted under the guidance of Professor N S Narayanaswamy, Mr. Jaimandeep Singh, and Ms. Anusha. The internship focused on building a foundational understanding of conversational AI and the underlying technologies that enable intelligent information retrieval and response generation.

The report is structured into two primary components. The first is a comprehensive literature review spanning the key concepts in the field: the evolution of conversational AI and chatbot architectures, the role of intent classification in dialogue systems, generative AI models and their architectures, real-world applications of AI agents in healthcare and education, natural language processing and understanding, and the principles of information retrieval — including sparse, dense, and hybrid approaches. The review concludes with an in-depth treatment of Retrieval-Augmented Generation (RAG), a framework that addresses several critical limitations of large language models by grounding their outputs in externally retrieved knowledge, along with a discussion of practical considerations when deploying LLMs in real-world settings.

The second component is a hands-on demo project that puts these concepts into practice. Using a custom dataset derived from publicly available content on core AI and NLP topics, the project implements and compares sparse and dense retrieval pipelines within an ElasticSearch environment. The experiments explore how BM25-based keyword retrieval and semantic vector search each perform across a range of query types, providing concrete insight into the trade-offs between lexical precision and semantic flexibility.

Together, the research and practical components of this internship offer a grounded introduction to the technologies driving modern conversational AI, with a particular focus on the retrieval mechanisms that make intelligent, knowledge-grounded systems possible.

3 Background and Research

3.1 Conversational AI in Chatbots

Unlike traditional approaches in which humans passively receive assistance from machines, collaboration in the age of AI involves active engagement and interaction between both parties. [**1**] Conversational AI is at the heart of this change in human-computer interaction. From virtual assistants to customer support bots, these systems aim to replicate real conversation. At its foundation, conversational AI enables machines to understand and respond to human language in a way that feels natural and contextually relevant.
Early chatbots relied on **template-based approaches** rooted in rigid rules and logic. They generated static text responses with minimal variability, often using placeholder variables to adapt slightly to user input. The introduction of **natural language understanding (NLU)** introduced basic comprehension abilities, but, overall, these systems remained limited in adaptability and nuance. In the early stages, **pattern recognition** was the primary technique, where systems would detect simple text patterns to interpret user queries and generate corresponding responses. Although this laid the groundwork for more advanced technologies, it also introduced notable limitations. Responses often felt **predictable and repetitive**, lacking contextual understanding or memory of previous interactions. Responses could easily become looping or robotic, revealing the need for more advanced techniques. These early systems were also often built as **monolithic architectures**, meaning all functional components were tightly interwoven. This design made it difficult to isolate or improve individual components without affecting the whole system, limiting scalability and flexibility. As chatbot technology evolved, natural language processing (NLP) and machine learning (ML) began to reshape capabilities, moving beyond fixed rules to more dynamic and intelligent systems. Traditionally, chatbot architectures were composed of four main components: graphical user interface, natural language processing, dialogue flow/management, and message abstraction layer.

Figure 1: Components to Traditional Chatbots: Modified from [**2**]

Modern systems, however, have moved away from rigid structures, embracing more flexible and **modular** designs. Natural language understanding (NLU) is the **only true AI component** within traditional chatbot architectures, responsible for detecting **user intent** and extracting relevant **entities** from the input. When combined with a graphical user interface (GUI), NLU plays a key role in defining the training data and managing the underlying model. Together, they form the **core infrastructure** that enables chatbots to understand input and respond intelligently. Within low-code or no-code graphical interfaces, dialogue flows are typically predefined using logic gates that map user input to specific outcomes. A **message abstraction layer** governs the responses of the bot at each dialogue turn, often using templates to personalize messages with dynamic content. As chatbot applications grow, maintaining this library of predefined responses becomes increasingly complex, especially when the content remains static. For questions outside the chatbot's domain or training, many systems employ **knowledge bases** or **semantic similarity searches**. They were early versions of what would later evolve into **retrieval-augmented generation (RAG)** pipelines.

Feature	Monolithic Approach	Modular Approach
Structure	Single, unified codebase or system	System composed of separate, independent components
Flexibility	Low, changes in one part can affect whole	High, individual components can be updated or replaced independently
Debugging & Testing	Difficult, hard to isolate specific issues	Easier, components can be separately tested
Adaptability	Rigid, often requires retraining the whole system for changes	Adaptable, components can evolve individually
Complexity	Simpler to implement initially	More complex design & coordination needed
Prevalence	Common, used in many LLMs due to training effectiveness	Less common but gaining interest for flexibility and maintainability
Scalability	Limited, hard to scale parts independently	Better, new components can be added without disrupting the whole
Transparency	Hard to understand internal logic	Better transparency on internal functions
Use Examples	GPT-style LLMs with end-to-end training pipelines	RAG architectures or plug-play modules

Table 1: Monolithic vs Modular Approach to Implementing Models

Ultimately, the future of AI may not be a competition between monolithic and modular approaches, but rather a symbiosis. By embracing this diversity and fostering collaboration between different AI paradigms, we can create a future where AI truly serves and enhances human capabilities across all domains of life. This hybrid approach could pave

the way for AI systems that are not only more capable but also more aligned with human needs and values. [**3**]
As the technology evolved, more advanced rule logic and natural language processing (NLP) made interactions smoother and more flexible. The real change occurred with the introduction of machine learning. This allowed these systems to learn from user inputs, understand intent more accurately, and respond with greater relevance. Deep learning took it a step further, helping systems build and refine a knowledge base over time, becoming smarter and more context-aware with each conversation. This progression from rigid, preprogrammed responses to adaptive, learning-driven systems marks a major leap in how machines interpret and engage with human language.
Conversational AI tools come in a range of forms depending on their complexity and purpose. At the most basic level, a chatbot is any software that simulates human conversation, often powered by rule-based decision trees and rigid menu navigation. These are found across websites, messaging apps, and support portals. AI chatbots go one step further by using technologies such as machine learning, natural language processing (NLP), and natural language understanding (NLU) to interpret user input and respond more intuitively. They can understand intent, handle more complex queries, and offer smoother user experiences with fewer misunderstandings. Virtual agents or intelligent virtual assistants are advanced versions of AI chatbots. They combine conversational AI and deep learning with tools like robotic process automation, allowing them not only to respond to questions, but also to take meaningful actions such as retrieving data, completing transactions, or interacting with enterprise systems. These systems make better use of the internal knowledge of a company and can integrate with existing infrastructure to provide intelligent end-to-end customer support.
One of the biggest advantages of conversational AI is its ability to provide 24/7 customer service, automate workflow, and eliminate long wait times. This significantly improves user experience and increases customer satisfaction, both of which play an important role in maintaining brand loyalty. By handling common queries and routine tasks, these systems also reduce the number of users who require human assistance, allowing businesses to scale support operations without proportionally increasing staff. In addition, conversational AI tools can offer valuable insights into customer behaviour, including buying patterns and engagement trends, which can help design future interactions and strategies.

3.1.1 Intent Classification

In modern chatbot systems, **intents** play a crucial role in defining how the system interprets and responds to user input. An intent categorizes the user's goal or purpose for a specific conversational turn. Within platforms like Dialogflow, each agent is defined by a set of intents, which collectively shape the nature of the conversation. When a user sends a message (called an end-user expression), the platform uses intent classification to match that message to the most appropriate predefined intent. Each intent typically consists of several components: training phrases, actions, parameters, and responses. **Training phrases** represent what users might say in natural language; these are used to train the underlying machine learning model to recognize similar input. ML techniques

also help expand this list beyond what is explicitly written, allowing for more flexible matching. Once an intent is triggered, an **action** can be executed; this could involve system-side logic such as triggering a back-end service or calling an API. **Parameters** are values extracted from the user's message at runtime, often used to fill spaces or make decisions within the dialog flow. Each parameter is tied to an **entity type** that defines how the data were interpreted. Entities can be **system-defined** (such as dates, names, or numbers) or **custom-defined** to suit domain-specific needs. For each entity type, developers define entity entries, a set of equivalent terms that helps the system recognize variations in user phrasing.

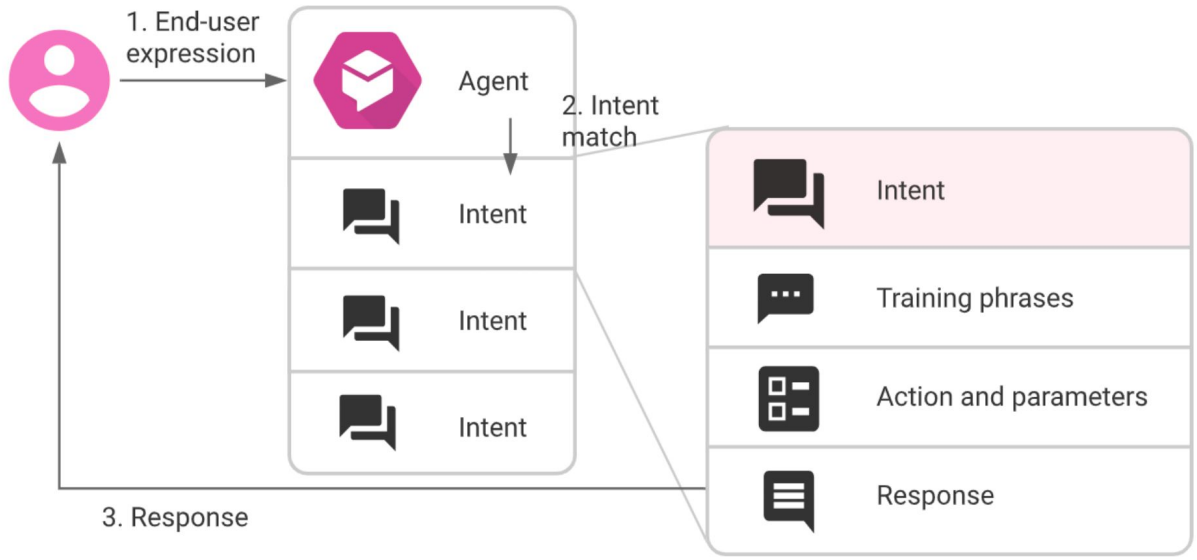


Figure 2: Intent Matching: Modified from [**4**]

The **responses** of an intent are the output messages sent back to the user. These may be in the form of text, audio, or even visuals, and they aim to either provide answers, collect more information, or gracefully end the interaction. More **complex intents** may also incorporate **contexts** and **events**, which allow the system to track the state of the conversation and respond appropriately to user behaviour over time. Behind the scenes, chatbot models rely on **sequence models**, especially in intent classification tasks. These supervised learning models handle sequential data, such as text or audio, and are trained to recognize patterns in input sequences to make predictions. Common applications include sentiment analysis, machine translation, and even creative tasks like music generation. Intent classification itself is a core part of chatbot intelligence. Using a combination of machine learning and NLP techniques, the model learns to associate user input with a predefined intent category. Once trained, the system can classify user queries in real time, allowing for fast and automated responses without human intervention. Over time,

with continuous data input and feedback, the model improves in accuracy and adaptability.
Intent classification itself is a core part of chatbot intelligence. Using a combination of machine learning and NLP techniques, the model learns to associate user input with a predefined intent category. Once trained, the system can classify user queries in real time, allowing for fast and automated responses without human intervention. Over time, with continuous data input and feedback, the model improves in accuracy and adaptability. Intent categories typically fall into six types:

1. **Informational:** Seeking answers or clarification.
2. **Support:** Addressing technical or product-related issues.
3. **Transactional:** Performing actions such as bookings or purchases.
4. **Feedback:** Giving opinions about the chatbot or service.
5. **Navigational:** Asking for directions or navigating platforms.
6. **Small Talk:** Casual, non-task-based conversation.

Training a chatbot for effective intent recognition involves several steps: defining intent categories that align with the chatbot's purpose and user needs, gathering a diverse dataset of messages, labelling them correctly, and training the model. Once deployed, regular monitoring and user feedback, such as ratings or reviews, should be integrated into the refinement cycle to continuously improve classification accuracy.
Real-world applications of intents are found across industries:

- In **e-commerce**, intents may involve product inquiries, order tracking, promotional offers, or sizing help.
- In **customer service**, intents can include password resets, complaints, billing support, and ticket creation.
- For **insurance**, typical intents handle policy details, claim filing, and renewal reminders.
- In **banking**, intents can cover account setup, balance checks, transaction history, and fund transfers.

Domino's and Dialogflow
Domino's Pizza in the US implemented Dialogflow to power their conversational ordering system, launching **"Dom"**, an AI-based online assistant within their mobile app. They were among the first in the food industry to introduce a native voice ordering assistant, using Dialogflow's intuitive user interface and scalable intent classification capabilities. With more than 50 years of customer service knowledge integrated in their system and Dialogflow's natural language understanding (NLU) strengths, Domino's was able to manage complex customer orders with ease. The Dialogflow platform proved to be easy to train and efficient to use, allowing Domino's to streamline user experience in diverse

ordering scenarios. *"We view conversational technologies as the next evolution in e-commerce, the future of the traditional e-commerce experience. Dialogflow helps us stay ahead of the curve."* - Mandi Galluch, Domino's Program Leader for Digital Experience [**5**]

Feature	**Chatbots**	**AI Agents**
Conversation Style	Follow scripted flows and predetermined intent.	Understand context, intent, and nuanced human language using LLMs.
Handling Complex Queries	Struggle with off-script or complex questions; may give irrelevant or repetitive answers.	Manage complex interactions smoothly with meaningful, context-aware responses.
Conversation Summaries	Cannot summarize or hand overly detailed interaction history.	Generate automatic conversation summaries and detailed handovers to human agents.
Autonomy in Issue Resolution	Limited; often requires escalation to human agents.	Capable of independently resolving issues, completing tasks, and wrapping up interactions with the efficiency of a human agent.
Setup and Maintenance	Requires technical expertise; resource-intensive training, and costly infrastructure	No coding needed; configurable with plain language prompts, making setup easier and scalable.
Industry Specific Knowledge	Struggle with industry-specific terminology or regulatory nuances, often leading to less reliable support in specialized areas.	Understands and responds using industry-specific language and complies with regulations.
Real-Time Decision Making	Poor handling of dynamic, transactional questions mid-conversation	Effectively combine contextual understanding with transactional details to deliver accurate, real-time responses without disrupting the flow of conversation.
User Experience	Can frustrate users due to rigid responses and limited understand	Offers more natural, human-like conversations, improving customer satisfaction significantly.

Table 2: Chatbots v/s AI Agents

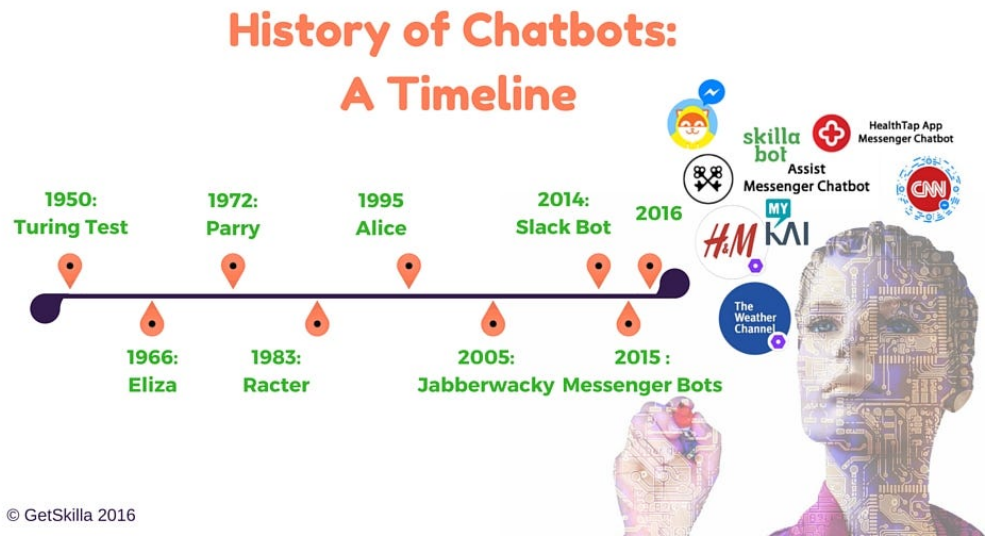


Figure 3: Evolution of chatbots

3.2 Generative AI in Chat bots

Generative AI has significantly advanced the capabilities of modern chatbots. Rather than relying on pre-written responses, these systems are now able to understand natural language. From casual queries to more complex, nuanced questions, they respond with context, clarity, and empathy. When integrated with an organization's knowledge base, generative AI does not retrieve answers; it generates them in real time, adapting to the user's specific intent. Generative AI refers to models capable of producing new and original content based on patterns learned from existing data. Upon receiving a prompt, these models generate unique and creative responses based on their training data. By combining **language learning models**, **machine learning**, and **neural networks**, these systems are able to simulate human-like creativity. These systems rely on deep learning algorithms and large datasets to understand relationships and structures within data. Using this understanding, neural networks generate content that aligns with the learned patterns. Generative models improve their performance through **iterative training**, adjusting internal parameters to reduce the difference between desired and generated results. Over time, this refinement enables them to produce high-quality and contextually accurate content.

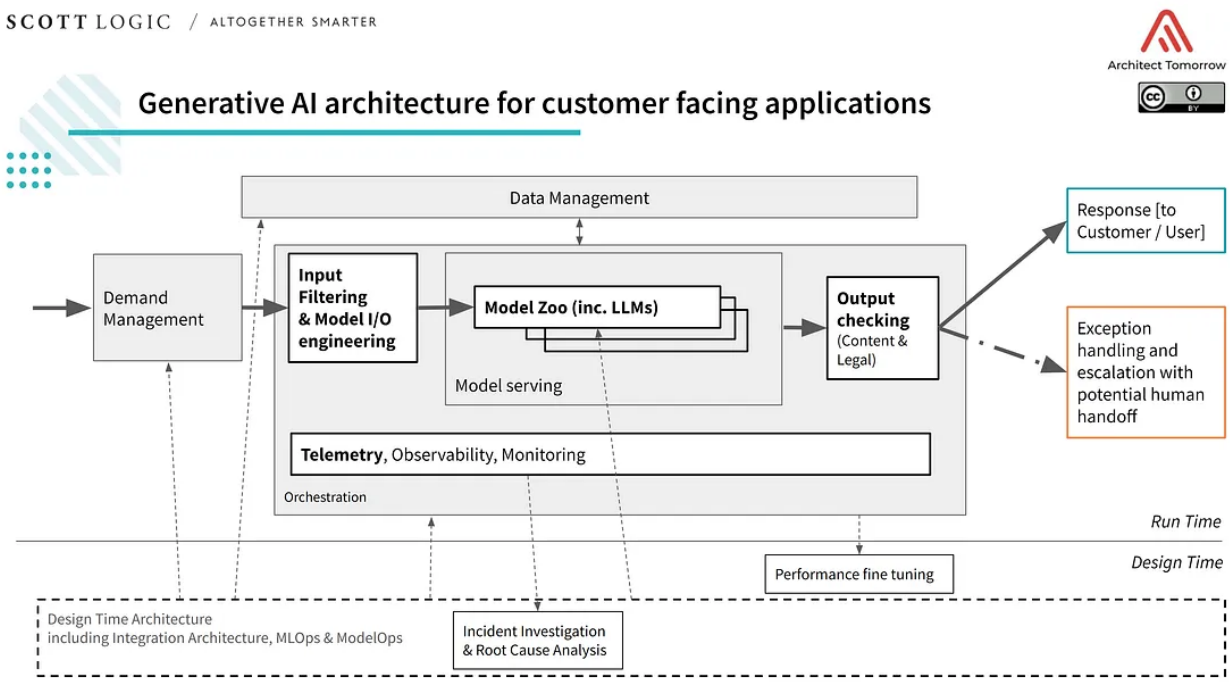


Figure 4: Generative AI for Customer Facing Applications from [**6**]

There are several core types of generative AI models. These models use various approaches to learn to produce new and realistic data, each with its own strengths and limitations. **Generative Adversarial Networks** consist of two components: a **generator** that creates data and a **discriminator** that evaluates its authenticity compared to real data. During training, the discriminator is fed a mix of real and fake images and learns to classify them correctly. Its feedback is then used to update the generator's parameters through **backpropogation**, guiding it to produce images that are increasingly difficult for the discriminator to distinguish from real ones. Over time, this adversarial dynamic pushes the generator to create highly realistic outputs. GANs are used mainly in fields that require image generation. GANs improve the functioning of the generator and the discriminator through **loss functions**. Generator loss is the negative of the discriminator's output when applied to fake samples. The discriminator loss is the summation of losses when it correctly classifies real and fake sample data. Deep Convolutional GANs (DCGANs) regularization methods and alternative loss functions have been employed to improve the stability of GANs while training.

A **Variational Auto Encoder** is a type of artificial neural network used primarily for unsupervised learning tasks, including image generation, text generation, and molecular design. Its goal is to generate new data points that resemble those in the training set by learning a compact latent representation of the input data. The VAE consists of two main components: an **encoder** and a **decoder**. The encoder network contains layers of neurons that reduce the dimensionality of the input data, mapping it to a probability distribution in a continuous **latent space**. This latent space serves as a lower-dimensional representation that captures the crucial features and attributes of the input data necessary for its reconstruction. The encoder outputs the parameters (typically mean and

variance) of this distribution, representing where a given data point is likely to be present in that latent space. The dimensionality of the latent space can be adjusted based on the complexity of the data and the desired level of data compression. Once the distribution is defined, a point from the latent space is passed through the decoder network. The decoder also attempts to reconstruct the original input by upsampling the latent vector back to the confines of the original data by passing it through the layers of neurons in its network. Training a VAE involves minimizing the loss function composed of two terms: the **reconstruction loss** and the **Kullback-Leibler (KL) divergence**. The reconstruction loss (frequently measured using mean-squared error) evaluates how nearly the reconstructed output matches the original input, while the KL divergence term regularizes the latent space to ensure continuity and structure. Optimization of this loss is typically performed using gradient-based techniques, which adjust the parameters of both the encoder and decoder networks to improve performance over time.

Large Language Models(LLMs) are usually trained on vast volumes of textual data drawn from diverse sources, including textbooks, websites, and social media. At the backbone of more LLMs lies **transformers**. First introduced in *"Attention is All You Need"* by Vaswani et al. (2017) [**7**], transformers discard recurrence mechanisms in place of a self-attention mechanism. This capability to model long-range dependencies is a major advancement over traditional sequential models such as LSTMs [**8**]. The transformer architecture typically follows an encoder-decoder format. The **self-attention** layers compute the attention weights for each pair of tokens, capturing the importance of each token in the input data. Since the architecture lacks sequential structure, positional encodings of the tokens are added to the input embeddings. **Multi-head attention** in which several attention computations run in parallel and are then combined and linearly transformed into a single output. Each block in the architecture also includes a **feed-forward neural network** that applies the same transformation independently to each token, allowing the extraction of deeper features. Residual connections and layer normalization are layered in to make training smoother and to keep gradients flowing well even in deeper models without vanishing. Since all computations in a transformer are done in parallel, training is more efficient and scalable. **Autoregressive models** such as PixelRNN, approach image generation like a sequential task. They model the conditional probability of each pixel based on the previously generated pixels, typically those to the left and above. This is similar to feeding the image pixels into a character-level RNN, which is designed to handle sequences by processing data one character at a time, but adapted to two-dimensional image data. The horizontal and vertical scan of the image allows the model to learn complex spatial dependencies between pixels.

Each of these models offers trade-offs. Variational Auto-encoders (VAEs) are well-suited for probabilistic modelling and can perform efficient Bayesian inference (*a method of statistical inference in which Bayes' theorem is used to calculate a probability of a hypothesis, given prior evidence, and update it as more information becomes available*
[**9**]). However, their generated samples often appear slightly blurred. GANs are known for producing high-quality sharp images, though they are difficult to train because of their unstable optimization dynamics. PixelRNNs are more stable and straightforward in their training process, with a strong likelihood score. However, they are expensive during sampling and do not easily yield compressed representations of images.

Diffusion models are a class of advanced machine learning algorithms that are used to generate high-quality data, especially in tasks such as image synthesis and text generation. They begin by adding noise to the data and then train a model to reverse that process, gradually reconstructing the original input. This ability to go from noise back to structured, meaningful content makes them effective at producing realistic and detailed outputs. They have a **two-phase structure**: the **forward diffusion process** and the **reverse diffusion process**. In the forward diffusion process, an input (ex: an image) is iteratively noised until it becomes unrecognizable. The reverse diffusion process then works to restore the original structure from the noisy version. The resulting model is flexible and capable of capturing complex details and patterns. This gradual degradation and reconstruction approach helps diffusion models produce outputs with a high degree of fidelity. Their applications are growing fast, from powering cutting-edge tools like **DALL.E 2**, **DALL.E 3**, **Sora**, **Stable Diffusion**, and **Midjourney** to advancing fields like medical imaging, autonomous navigation, and personalized AI assistance. [**10**]

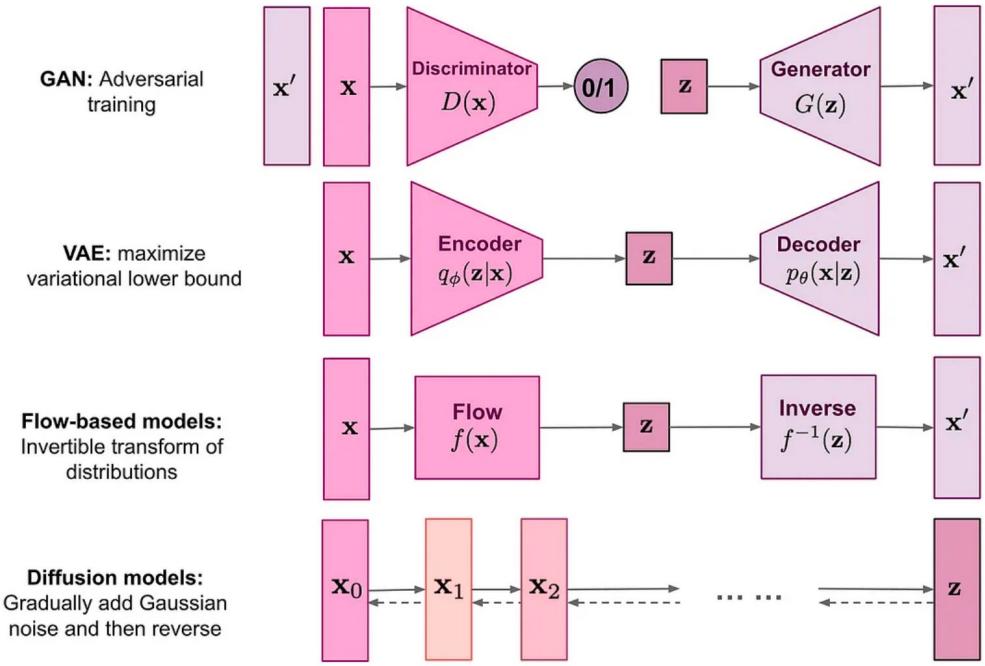


Figure 5: Types of Gen AI Architectures: Modified from [**11**]

Generative AI offers numerous advantages. It can automate content creation in industries such as advertising, entertainment, and marketing. It can also assist in problem solving tasks by generating innovative solutions or analyzing data to improve decision making. In software development, it helps streamline code writing and optimization. Generative AI chatbots and virtual assistants improve customer support by reducing response times and allowing human resources to be used for other purposes. Despite its benefits, generative AI presents challenges. Data bias remains a key concern, especially when the outputs reflect the existing prejudices present in the training data. Ethical risks such as misuse, disinformation, and job displacement also require scrutiny. Furthermore, gen-

erative models can produce unreliable or nonsensical outputs, especially when lacking domain-specific understanding.
In the future, generative AI is expected to shape research and innovation. In healthcare, for example, it can support drug discovery by generating and testing molecular structures. In industry, organizations are customizing generative AI systems trained on proprietary data to improve operations, logistics, and communication. Its potential impact spans from environmental initiatives, such as ocean cleaning robots, to productivity enhancements in everyday applications like spreadsheets, calendars, and email platforms. [**12**]

3.3 Real Life Implementations of AI Agents

AI agents are already being used in all industries in positive ways. They help automate complex tasks such as data gathering and analysis, help in decision making, and support personalized customer interactions. In retail, they predict demand and adjust pricing; in education, they adapt learning experiences to individual needs. Chatbots, virtual assistants, recommendation systems, and real-time monitoring tools improve both efficiency and user experience in sectors such as healthcare, finance, manufacturing, and transport.

3.3.1 AI in Health-Care

Artificial intelligence is transforming the future of medicine, particularly in the space of precision therapeutics, synthetic biology, and healthcare delivery. By applying AI tools to complex multimodal datasets, covering genomics, imaging, and clinical records, we may soon gain deeper insights into the cellular mechanisms of disease. This enables more targeted prevention strategies, especially through fields such as immunology, which could personalize diagnostics and treatment plans. The potential for impact is immense, particularly for cancer care, neurology, and the treatment of rare diseases, where individualized care could become the new norm.
Medical diagnostics involves evaluating conditions or diseases by analyzing a patient's medical history and test results to determine the underlying cause and ensure accurate treatment. Traditional diagnostic methods include imaging technologies (such as X-rays, CT scans, and MRI), blood tests, and biopsies. The information obtained through these tests is essential not only for initial diagnosis, but also for selecting the most effective treatment plan, monitoring disease progression, and evaluating the effectiveness of treatment over time. Performing timely diagnostics can help identify potential complications before they become serious. However, traditional methods alone can be time-consuming and prone to human error. Artificial intelligence has the potential to significantly improve this process by improving prediction accuracy, increasing speed, and improving the overall efficiency of diagnostics. To help healthcare professionals identify conditions with greater precision, **AI algorithms** can analyze vast amounts of medical data, including images, physiological signals, patient vitals, historical records and laboratory test results. The use of multiple data sources, also known as **multimodal data**, helps reduce the risk of misdiagnosis and supports the better management of chronic diseases through continuous monitoring. **Quantum AI** is an emerging area that promises to accelerate model training

and dramatically **improve diagnostic speed**. With processing capabilities far beyond classical computers, quantum AI could enable real-time analysis of enormous datasets, allowing for more efficient and accurate decision making. This could lead to faster diagnostic results and better patient care. **General AI** capabilities are being explored in **OpenAI's DeepQA**, **IBM's Watson**, and **Google's DeepMind** These systems aim to offer healthcare providers deeper insights into patient data, enhance clinical decision-making, and optimize treatment recommendations. By combining learning from diverse datasets with advanced reasoning, general AI could further streamline and personalize the diagnostic process. Despite promise, AI-driven diagnostic tools face several challenges. Effective algorithms require access to high-quality, labeled data, which is often difficult to obtain due to fragmented, incomplete, or unavailable medical data. In addition, biased training data can lead to algorithms that underperform for certain demographics, resulting in unfair or even harmful results. Ethical concerns also arise, especially when general AI systems are used on private or sensitive data sets. The issues of data privacy, algorithmic transparency, and accountability must be carefully addressed. Furthermore, many AI-based diagnostic tools are developed in isolation, lacking shared protocols or standards for interoperability. For AI to be reliably integrated into medical diagnostics, the field must focus on developing robust frameworks for collaboration and regulation. In general, analyzing medical diagnostics through the lens of AI remains an active and open research domain. It offers enormous potential in identifying hidden patterns in data and uncovering complex relationships in medical data, making diagnosis faster, smarter, and more accessible. [**13**]

Triage is a critical clinical protocol used in emergency and outpatient medical settings to prioritize patient care according to the urgency and severity of the patient's condition. Upon arrival, patients are initially screened by trained healthcare personnel, typically triage nurses, who collect demographic and symptomatic information and assess vitals and acuity levels. This process ensures that individuals in life-threatening conditions receive immediate attention, while those with non urgent needs are managed accordingly. Accurate triage not only improves patient outcomes, but also optimizes resource utilization in high-pressure environments where time and manpower are limited. However, the traditional triage workflow is often hindered by system inefficiencies, such as staff shortages, delayed assessments, and misclassification of patient urgency. These gaps can result in treatment delays, overcrowding, and increased risk of adverse events. To address these challenges, AI-powered systems are being integrated into the triage pipeline. Expert systems simulate the reasoning of human specialists by structured knowledge representations, while chatbots equipped with NLP techniques engage directly with patients to collect preliminary data and provide real-time guidance. By automating aspects of the triage process, these tools can accelerate initial assessments, support decision-making, and reduce the mental burden on medical personnel. Such intelligent systems offer significant potential to enhance the accuracy, speed and, scalability of clinical triage, ultimately leading to more efficient patient care delivery in both emergency situations and routine medical check ups [**14**] [**15**].

In **drug development**, AI is already showing promise. Systems like **DeepMind's AlphaFold** have transformed the prediction of protein structures, setting the stage for more efficient drug discovery. AI can improve clinical trial design, improve drug manufacturing,

and optimize complex processes that were once slow, costly, and prone to error. Over time, artificial intelligence will help researchers better predict which compounds are likely to succeed earlier in the pipeline - potentially reducing the financial and human costs of late-stage failures. Developments in **synthetic biology**, such as CRISPR gene editing and personalized cancer therapies, have been revolutionary. However, their production remains expensive and inefficient. With greater access to biological data, AI can help manage and decode their complexity. This will not only accelerate the creation of new therapies, but also reduce their cost and expand access worldwide, making cutting-edge treatment more unbiased. Long-term healthcare professionals will increasingly rely on AI not as a replacement but as an addition to their own expertise. Digital twins (virtual replicas of patients) could be used to simulate treatment effects before human intervention. This kind of AI consultation can allow clinicians to provide more personalized, safe, and efficient care. Current AI systems are proving to be useful in automating repetitive tasks and improving precision diagnostics despite their lack of intuitive judgment. In the next 5-10 years, advanced algorithms that require less training data can learn from unlabeled inputs and combine information across domains. Over the next decade and beyond, AI systems will evolve to become more intelligent and adaptive. Healthcare institutions are expected to transition to actively shaping custom AI platforms with tech partners. This vision of AI-augmented healthcare promises better outcomes for patients and more sustainable systems in general. [**16**]

3.3.2 AI in Education

The integration of artificial intelligence into educational systems is redefining teaching and learning by enabling personalized and adaptive experiences. Traditional classroom methods are being supported or even replaced by systems that dynamically respond to each learner's needs. Technologies such as machine learning, predictive analytics, and natural language processing (NLP) have allowed these systems to analyze cast datasets, including learner profiles, performance metrics, and engagement patterns to provide personalized content in real time. Adaptive learning systems leverage AI to modify teaching styles based on the learner's understanding. Machine learning algorithms identify trends and predict the learner's behaviour to adjust the pace and complexity of content. The key components of such systems include learner modeling, content customization, and feedback loops. Learner modeling creates detailed profiles using data from assessments, progress tracking, and behaviour. AI then uses these profiles to prioritize concepts or repeat material, ensuring the success of the learner. In addition, platforms that use NLP enable more natural interactions between learners and systems. Modern chatbots support contextual learning and make educational technology more intuitive. Together, NLU and NLG allow these systems to "understand" student queries and respond in human-like ways. The benefits of AI-powered adaptive are significant. Learners receive personalized instruction according to their specific pace, style, and proficiency. This level of personalization promotes greater retention and engagement. Educators gain insight through dashboards and analytics, helping them identify ineffective strategies or slower learners. For example, platforms such as reflective tutors and web-based systems such as ActiveMath and ELM-ART have demonstrated the ability to adapt teaching styles, catering

to different types of learners. However, the adoption of AI in education is not without challenges. Ethical and practical concerns must be addressed. It is vital to ensure that AI supports and does not replace educators. Data quality and domain specificity also pose obstacles to accurate personalization. To mitigate these risks, transparent design, rigorous testing, and continued involvement of educational experts are essential. [**17**]

3.4 Natural Language Processing

Natural language processing , or NLP, is a crucial area within artificial intelligence that enables machines to interpret, understand, and generate human language. It sits at the intersection of computer science, linguistics, and artificial intelligence and belongs to a broader field known as computational linguistics. The goal of NLP is to bridge the gap between human communication and machine understanding, so that computers can interact with people in ways that feel natural and intuitive. The development of NLP has a rich history that spans several decades. In the 1950s, Alan Turing proposed the now-famous "Turing Test" to assess a machine's ability to exhibit human-like intelligence, using language as the metric. Early NLP systems in the 1950s and 1960s were heavily rule-based relying on manually written sets of instructions and grammar rules. This approach was effective in narrow domains, but struggled with complexity and ambiguity. By the 1970s, the field had shifted toward statistical methods, using probabilistic models to learn patterns in language data. In the 1980s and beyond, particularly through more efficient algorithms and larger datasets. The emergence of deep learning and transformer architectures has completely transformed NLP models. Models such as BERT, GPT, and their successors have demonstrated excellent language understanding and generation abilities utilizing deep neural networks and the self-attention mechanism [**18**]. These models no longer rely on manually coded grammar rules or even simplistic statistical methods. Instead, they learn complex language structures from massive amounts of data, making them more flexible, accurate, and scalable. Today, NLP encompasses a range of techniques, including rule-based systems, statistical methods, machine learning, and deep learning approaches. Rule-based NLP still finds use in some domain-specific or deterministic applications. Statistical NLP helps to make predictions and generalizations from language data. However, the most robust systems now use hybrid methods that combine rule sets with machine learning to create flexible and efficient pipelines. A standard NLP pipeline involves several steps. First comes **data processing**, where the raw language input is cleaned and structured. This stage involves tokenization (breaking text into words or phrases), stop word removal (removing common but low-information words), stemming and lemmatization (reducing words to their root forms), part-of-speech tagging, and syntactic parsing. Next is **algorithm development**, which involves building models for specific tasks such as sentiment analysis, named entity recognition (NER), topic modeling, machine translation, and language modeling. Each of these tasks has unique challenges and requires careful tuning of data, architecture, and training techniques. One of the key strengths of modern NLP is the diversity in its application. From spam filters to real-time translation and intelligent assistants, NLP is embedded in everyday tools. It improves communication, automates content organization, powers chatbots, and supports acces-

sibility efforts such as speech-to-text services.

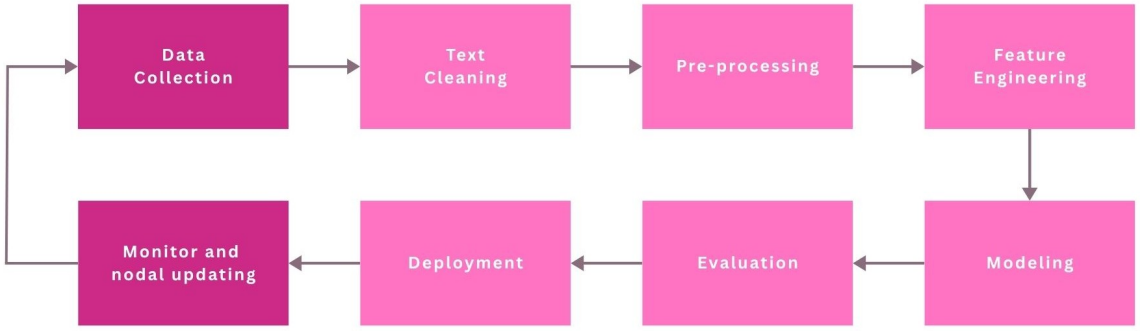


Figure 6: NLP working: Modified from [**19**]

Despite all the progress, NLP still faces fundamental challenges. Human language is inherently ambiguous and filled with nuance, which is difficult for machines to parse. Models also struggle with homonyms, inconsistent syntax, and slang, all of which vary across languages and communities. Multilingual applications are especially tricky, since NLP models typically need to be trained or fine-tuned on a specific language. Language identification becomes a necessary first step before processing multilingual content. Ensuring data quality and domain specificity remains a challenge. General-purpose language models may perform poorly in specialized fields such as law, medicine, or finance unless retrained on industry-specific data. Unspecific or shallow training data limit the ability of the model to understand deeper meaning. Privacy and ethics also play a role. NLP applications often involve processing sensitive data. This raises concerns about data security, consent, and algorithmic bias. As such, the implementation of NLP in high-stakes industries must be approached with care. The future of NLP is deeply intertwined with the evolution of generative AI. Tools like ChatGPT and DALL-E have reshaped AI's capabilities. In the coming years, NLP is expected to become more **personalized**, with systems learning to tailor responses based on the language habits, preferences and tone of the user. Integration with other AI disciplines, such as speech recognition, computer vision, and robotics, will result in multimodal systems capable of a deeper contextual understanding. NLP models that can interpret voice, gesture, and visual cues alongside text are becoming a real possibility. More multilingual capabilities will also emerge, especially as demand grows in international markets. Perhaps most importantly, NLP systems are learning to sound more human. They are becoming better at understanding nuance, maintaining tone, and carrying out natural-sounding conversations.

Platform	Strengths	Limitations	Ideal Use Case
Azure Conversational Language Understanding (CLU)	Multilingual support (96 languages); high semantic accuracy with less training data; built-in intent/entity recognition	Limited community resources; lacks support for some legacy LUIS features (e.g., nested ML entities); requires migration from LUIS	Enterprise applications requiring multilingual NLU; organizations transitioning from LUIS before its deprecation
Google Dialogflow	Supports both text and voice interfaces; multi-channel integration; good for structured data like dates/currencies	Limited advanced customization; fewer enterprise integration features than some competitors	Scalable chatbots and assistants on multiple platforms; devs needing ease of deployment with wide language support
Amazon Lex	Deep learning-powered NLU and speech recognition; seamless integration within AWS ecosystem	Limited third-party integrations; higher setup complexity for non-AWS users	Conversational interfaces in customer service and virtual assistants built within AWS environments
IBM watsonx Assistant	Strong enterprise NLP tools; integration with web, mobile, and social platforms; multilingual support	Less intuitive development environment; lacks visual builder tools for conversation flow	Secure, scalable enterprise chatbots across industries requiring advanced language processing
Wit.ai	Free for commercial use; strong intent recognition and speech-to-text capabilities; easy onboarding	Limited enterprise-grade features; lacks built-in integrations with popular platforms	Lightweight NLP apps, voice interfaces, and early-stage startup solutions

Table 3: Comparison of Specialized NLP Platforms

3.4.1 Natural Language Understanding

Natural Language Understanding is a specialized subfield within NLP that focuses on interpreting meaning and intent behind text or speech. NLU transforms unstructured data into structured insights by grasping the semantics behind it. NLU systems typically respond to supervised or unsupervised learning. In supervised learning, models are trained on annotated examples to help them learn how to classify intents or extract entities. Unsupervised learning helps in discovering hidden patterns and relationships in large volumes of unstructured data, without relying on labeled datasets. The general flow in an NLU system begins with **preprocessing**, which includes steps such as tokenization and part-of-speech tagging. The models then analyze the **structure** of the input by mapping user intents, extracting key entities, and building semantic relationships. This structured understanding is used to trigger appropriate actions or generate accurate responses.
NLU powers a variety of applications. In marketing and sales, chatbots use NLU to qualify leads by understanding what a user is looking for. Voice assistants rely on it to interpret spoken commands and perform tasks. Customer service bots use NLU to detect frus-

tration or satisfaction in messages and adjust their responses accordingly. Email sorting tools use NLU to classify messages by intent, priority, or content. Similarly, text analytics platforms rely on NLU to analyze customer feedback, detect trends, and summarize long-form input. Across industries, the ability to turn messy human language into actionable data has massive operational value. The core components of NLU include

- **Tokenization:** breaking down text into smaller units for processing
- **Part-of-Speech Tagging:** labeling each token with grammatical roles
- **Named Entity Recognition (NER):** identifying people, places, organizations, etc.
- **Intent Classification:** determining the user's goal or purpose
- **Dependency Parsing:** analyzing grammatical structure and relationships
- **Contextual Analysis:** evaluating inputs based on broader conversational or situational context

Most modern NLU systems rely on transformer-based models to handle these steps efficiently. transformers can capture long-range dependencies in text, making them ideal for grasping meaning in complex sentences or dialogues. Embeddings are used to map the relationships between words in vector space, helping the model understand similarity, relevance, and sentiment. Modern chatbots rely heavily on natural language understanding (NLU), a core subfield of natural language processing (NLP), to discern user meaning and respond appropriately. NLU enables systems to interpret input not just at the surface level, but with an understanding of context, grammar, and intent. It allows machines to read and comprehend natural language the way humans do by mapping textual or spoken inputs to specific intents that guide the next steps, whether it is answering a question, taking an action, or continuing a conversation meaningfully.
By combining deep learning, transformer-based models, and attention mechanisms, modern systems can now handle tasks like intent recognition and response generation with significantly higher accuracy than previous rule-based or purely statistical models. This fusion of comprehension and generation is what powers today's chatbots, from simple support agents to complex assistants like ChatGPT.

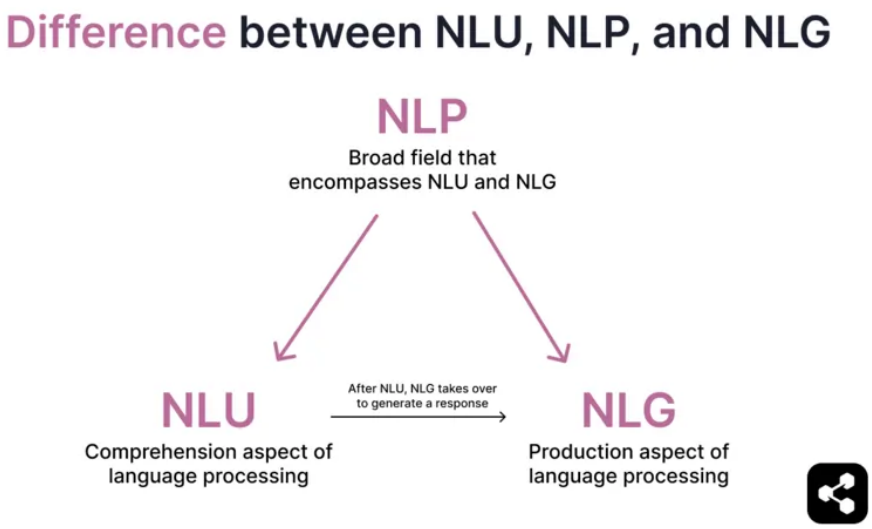


Figure 7: Difference between NLU, NLP, NLG from [**20**]

3.5 Information Retrieval

Information Retrieval (IR) refers to the process of identifying, locating, and delivering relevant information from large, often unstructured data sets in response to a user query. It is designed to handle **natural language input** and return results based on **relevance**, not just exact matches. Unlike traditional database searches, which rely on structured queries and a fixed structure of data, IR systems can interpret loosely phrased input and offer results that align with user intent. IR plays a vital role in modern applications, from search engines to virtual assistants, to recommendation systems, and enterprise search tools, by enabling users to access valuable information efficiently and intuitively.

The concept of IR has roots in *early library science*, in which efforts were made to catalog, index, and alphabetize information for easier access. In the 1880s, punched cards began to be used for processing data, marking an early attempt to automate information management. In 1931, Emanuel Goldberg developed the **"Statistical Machine"**, one of the first electromechanical devices capable of retrieving information from a microfilm. This marked a pivotal shift toward automating search. The mid-20th century saw IR evolve into a formal scientific discipline with the rise of computing. Pioneers like Hans Peter Luhn and Gerard Salton shaped its foundation, especially through the development of the **SMART information retrieval system** in the 1960s. SMART introduced major concepts like the term-document matrix, vector space model, relevance feedback, and the Rocchio algorithm, all of which still influence IR systems today. These innovations paved the way for future developments in automated indexing and retrieval systems, especially as computing power grew. By the 1990s, IR technologies moved the field from research laboratories to a mainstream global infrastructure.

IT models offer different frameworks for how systems interpret queries and match them with documents. Classical models serve as foundational approaches, while non-classical and alternative models have emerged to handle more nuanced challenges, often incor-

porating modern technologies like machine learning and large language models.

1. The **Boolean model** treats documents as sets of terms and uses logical operators (and/or/not) to match documents. Although simple and deterministic, it lacks the ability to rank documents based on their relevance.

2. The **probabilistic model** estimates the probability that a document is relevant to a given query, factoring in term frequency and document length. It enables ranked results, making it better suited for large-scale search.

3. **Latent Semantic Indexing (LSI)** introduced a deeper understanding by using singular value decomposition to uncover relationships between terms and documents. This allows the system to capture context and semantic meaning, linking conceptually related texts that might not share exact items.

An IR process is typically triggered by a user query. The system first builds an index of the available documents by analyzing and extraditing key terms. When a query is submitted, it is parsed and compared with the indexed content. **Ranking algorithms**, such as **BM25**, assign scores based on relevance factors such as keyword frequency, document length, and semantic overlap. The results are then presented to the user, typically in order of relevance. One major difference between IR and traditional database search is ranking. Database searches return exact matches, while IR returns graded relevance. An IR system usually consists of several key modules:

1. **Document Collection:** the body of data to be searched

2. **Indexing Engine:** processes documents and maps key terms to content

3. **Query Processor:** interprets the user's input and reformulates it for matching

4. **Ranking Algorithm:** scores documents based on relevance

5. **User Interface:** users interact with the IR model by inputting queries and viewing results through this interface.

Information retrieval plays a critical role in virtually every domain. From aiding researchers in digital libraries to powering product recommendations in e-commerce, IR systems support knowledge discovery and decision making on both local and global scales. In healthcare, they help locate patient records or treatment protocols; in legal technology, they search through vast archives of case law. Search engines rely on IR models to deliver relevant and ranked results, while personalized systems adapt based on user behaviour and preferences. IR acts as the bridge between humans and massive digital knowledge bases.
IR systems save time and reduce effort by providing fast, targeted access to information. They also uncover patterns and trends that support knowledge discovery. Personalization allows users to receive results tailored to their preferences, and professionals in all fields use IR for decision support and research. Natural language is inherently ambiguous. Queries can be vague or context-dependent, making interpretation tricky. Relevance

itself is subjective; what is useful to one user may not be useful to another. Semantic gaps between how people phrase things and how systems represent text can lead to mismatches. As data volume increases, scaling IR while maintaining quality becomes another challenge.
Machine learning and Generative AI are reshaping IR. Systems are learning from user interactions, adapting to context, and improving semantic understanding. Instead of returning a ranked list, modern systems increasingly generate direct, context-sensitive responses. Evolving natural language processing techniques are helping bridge the gap between intent and text representation. It is moving from a reactive tool to a more intelligent and conversational experience.

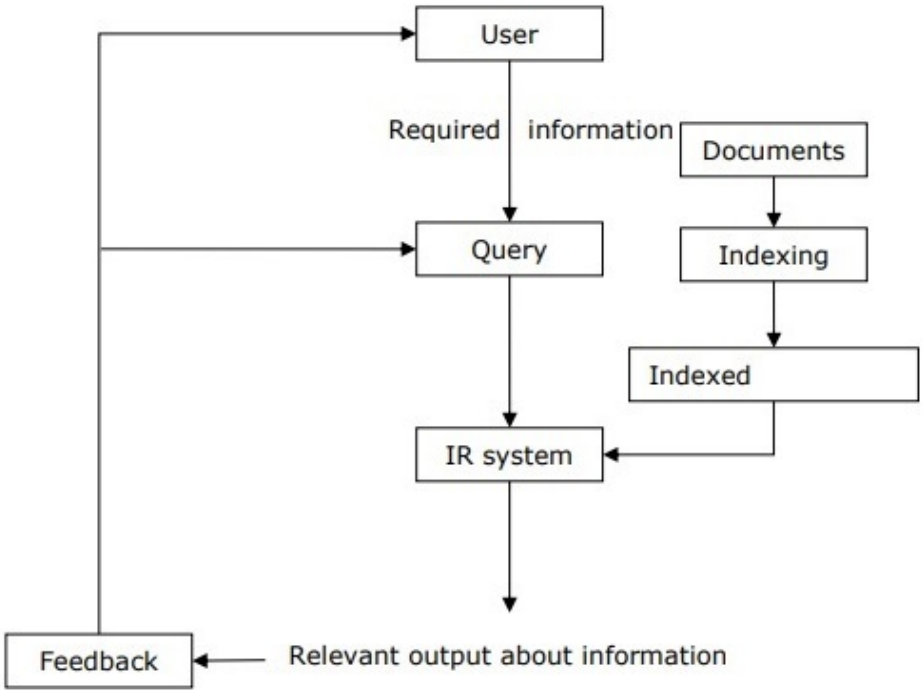


Figure 8: Information Retrieval Pipeline: Modified from [**21**]

3.5.1 Sparse Retrieval

Sparse retrieval methods form the backbone of traditional search systems, relying on the frequency of terms and their distribution across documents. In sparse retrieval, documents are represented as texts reflecting the counts of each word present in a predefined vocabulary. Since the vocabulary is often extremely large and any given document contains only a small subset of words, these vectors are typically sparse; i.e., most entries are zero. An inverted index is used to efficiently track which documents contain specific words or phrases from a user's query.
A key component of sparse retrieval is the use of **Term Frequency - Inverse Document Frequency (TF-IDF score)**. Term Frequency (TF) captures how often a term appears in

a document, while Inverse Document Frequency (IDF) reflects how important that term is based on how rarely it occurs across the entire collection of documents. Combining TF and IDF yields a weighted score that highlights terms that are frequent within a document and rare across the corpus, thus helping to improve retrieval performance. However, TF-IDF does not account for synonyms or deeper meaning, which can limit its usefulness for semantically rich queries.
One of the most widely used algorithms for sparse retrieval is **Okapi BM25**. Developed in the 1980s and 1990s at the City University of London, BM25 is a probabilistic model that refines the TF-IDF scoring by introducing **document length normalization**. This method prevents longer documents from dominating the rankings. BM25 also employs a mechanism known as **saturation effect**, ensuring that very frequent terms contribute only up to a limit, rather than inflating relevance scores indefinitely.
BM25 remains popular in real-world applications due to its consistent performance, high reliability, and computational efficiency. However, BM25 has limitations. It operates purely on a lexical level and lacks semantic understanding. It also does not employ embeddings and, therefore, cannot capture deeper relationships or meanings between words. Typical applications of BM25 include search engines, question-answering systems, recommendation systems and legal & medical text retrieval. Despite its limitations, BM25 is often combined with dense retrieval techniques in hybrid systems to enhance retrieval performance. It remains a go-to method for benchmarking baseline performance in many academic and industrial retrieval systems. [**22**]

3.5.2 Dense Retrieval

Dense retrieval refers to a modern approach that represents both documents and queries as dense neural embeddings, continuous vectors that capture meaning, rather than just individual keywords. These embeddings are often generated by transformer-based models like BERT, or Sentence-Transformers, which encode semantic context into compact representations. The retrieval is then performed based on **vector similarity** (using cosine similarity or a dot product), allowing systems to match the intent of the user more intelligently than using purely keyword-based methods.
A typical dense retrieval system uses a *dual-encoder* setup:

- Query encoder: transforms incoming queries into dense vectors
- Document or passage encoder: converts each text segment into its own vector

These models may be jointly fine-tuned so that relevant query-document pairs are close in vector space. Sparse models respond on inverted indices and literal keyword matching, which struggle with synonyms or paraphrases. Dense models excel at capturing underlying meaning and can surface relevant results even when exact terms do not match.

Feature	(Sparse Retrieval)	**Dense Retrieval**
Representation	Sparse, keyword-based	Dense, semantic embeddings
Keyword mismatch	Prone to errors	Handles synonyms and paraphrasing
Context understanding	Limited	Deep understanding via pre-trained models
Computation Cost	Low	High - inference and embedding are resource-intensive

Table 4: Advantages over Sparse Retrieval [**23**]

Documents are usually split into smaller chunks to preserve context and ease relevance matching. Techniques like overlapping chunks or syntax-based splitting maintain coherence while increasing the precision of retrieval. These chunks are then encoded into embeddings and stored in vector databases. Search is enabled via approximate nearest neighbour (ANN) methods to efficiently retrieve the most similar chunks at the time of inference. Dense models are typically trained using **contrastive learning**, where positive and negative pairs teach the system which embeddings should be closer or farther apart. Techniques like in-bath negatives, domain-specific pretraining, and pseudo-labeling further refine performance. [**24**]

3.5.3 Hybrid Retrieval

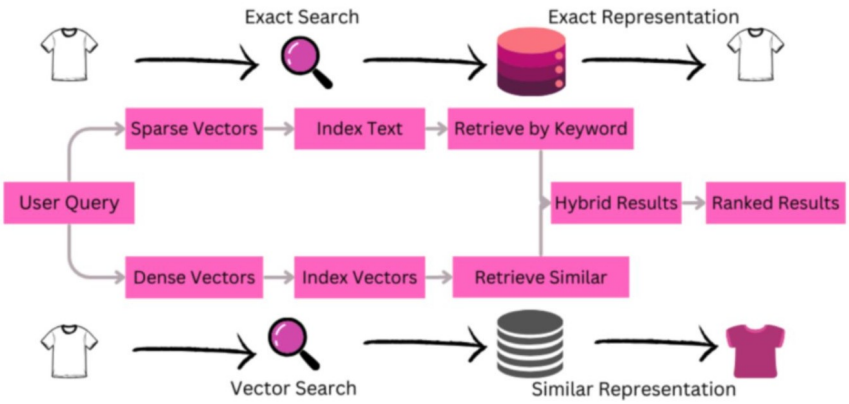


Figure 9: An Outline of Hybrid Search: Modified from [**25**]

Since sparse and dense searches excel in different areas, hybrid search combines the strengths of both sparse (keyword-based) and dense (embedding-based) retrieval methods to deliver more accurate and contextually relevant results. The search process typically involves two steps: candidate retrieval and relevance ranking. First, the search processes the query to generate embeddings and uses them to retrieve a broad set of

relevant candidates. Both sparse and dense retrieval outputs are merged into a single-ranked list using fusion algorithms, such as reciprocal rank fusion, which weigh the rankings from each source and penalize lower-ranked results. This hybrid strategy allows systems to handle a wide range of user queries, from exact keyword searches to more abstract or multilingual input. It also reduces the burden of perfectly phrased prompts by offering flexible retrieval, ultimately improving user experience and system robustness. Reciprocal Rank Fusion (RRF) is a rank aggregation technique used to combine multiple ranked lists into one single and unified ranking. It works by assigning each document a score based on its position in the individual rankings. The RRF score is calculated using the formula:

```
RRF(d) =  1 / (k+r(d))
where,
k is a constant (typically 60)
r(d) is the rank of the document
d is the document being ranked
```

In retrieval-augmented generation, RRF helps integrate results from sparse, dense, or hybrid retrievers, leading to a more balanced document set to generate final responses from. This prevents either sparse or dense rankings from overpowering the final results.

Feature	**Sparse**	**Dense**	**Hybrid**
Core Principle	Keyword based search	Semantic search	Semantic search
Applicable for	Fully/semi structured data	Unstructured data	Both fully/semi structured data & unstructured data
Preferable with	Advanced search filters	Natural language interface	Both advance search filters and natural language interface
Best suited to capture	Explicit content in the query	Implicit content in the query	Both implicit and explicit content in the query
Frameworks	ElasticSearch, Solr, Pinecone	FAISS, Vespa, Solr, Pinecone	Solr, Pinecone

Table 5: Sparse Search vs Dense Search vs Hybrid Search: Modified from [**26**]

3.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation is a powerful framework that combines information retrieval with text generation to produce responses that are both accurate and contextually relevant. This architecture allows LLMs to generate more accurate, informed, and context-aware information from external knowledge sources rather than relying solely on internal parameters or training data. The RAG pipeline is structured around three key stages: indexing, retrieval, and generation. Each stage plays a distinct and interconnected role in

ensuring the overall performance of the system.
The pipeline begins with the indexing stage, where raw textual data is prepared and structured in a way that supports efficient retrieval. This involves converting the input documents into embeddings, which are dense numerical representations of text that capture its semantic meaning. These embeddings are created using models trained to understand the relationships between words, phrases, and ideas, allowing similar texts to be placed closer together in a high-dimensional vector space.
To enhance both the precision and efficiency of retrieval, documents are typically chunked before embedding. Chunking refers to the splitting of longer documents into smaller, meaningful segments called chunks that can be individually indexed and retrieved. This is essential because it ensures that only the most relevant parts of a document are considered during retrieval. In practice, chunking improves search performance, retains relevance to user queries, and maintains the context in responses. Once chunked and embedded, these vectors are stored in a vector database, a specialized system optimized for similarity-based searching. Unlike traditional databases that rely on exact matches, vector databases allow for approximate nearest-neighbour searches, which prioritize relevance based on semantic similarity rather than surface-level matching.

LLM Limitation	**How RAG Addresses It**
Knowledge can be outdated	Retrieves up-to-date information from external sources in real time
Prone to hallucinating facts	Grounds responses in actual data, reducing chances of fabricated answers
Relies on static training data	Supplements generation with dynamic, real-world context
Costly and limited retraining	Taps into external knowledge without needing extensive retraining
Struggles with precision and long-term context	Retrieves only the most relevant chunks to improve accuracy and coherence

Table 6: How RAG Mitigates Common LLM Limitations

When a user submits a query, the retrieval stage is activated. The query itself is also converted into an embedding using the same model that was used to encode the documents. This ensures that both the query and the document chunks exist in the same vector space and can be meaningfully compared. To identify the most relevant pieces of information, the system calculates similarity scores between the query vector and the stored document vectors. One of the most widely used techniques for this purpose is **cosine similarity**, which measures the cosine of the angle between two vectors. Rather than focusing on their magnitude, cosine similarity evaluates the direction in which the vectors point, capturing how similar they are in meaning. A score of 1 indicates perfect similarity (same direction), 0 indicates no relation (orthogonal vectors), and -1 suggests complete opposition in meaning.

Technique	Description	Best Use Case
Fixed-Size Chunking	Break by tokens	Similarity, predictability
Sentence-based	Break by sentence boundaries	Natural coherence
Recursive	Tries longer spans, then breaks down	Maintains context
Semantic Chunking	Break by topic shifts	Deep search accuracy

Table 7: Chunking Strategies

These similarity computations allow the system to rank document chunks and retrieve only those that are most aligned with the query's intent. Techniques like **k-Nearest Neighbours (kNN)** are commonly used to retrieve the top k closest vectors. For larger or more complex retrieval systems, Hierarchical Navigable Small World (HNSW) graphs are employed to speed up the search process while maintaining high accuracy. HNSW allows the system to scale to millions (or even billions) of documents without significant performance degradation. The result of this stage is a small, curated set of relevant chunks that provide the most useful information in response to the query.
In the final generation stage, the retrieved content is passed to a large language model, which uses it as a context to craft a fluent and informative response. A key factor influencing the quality of the responses generated is the design of the prompt. In RAG systems, a prompt typically includes three components:

- **Context:** The retrieved chunks serve as factual or conceptual background.
- **Query:** The user's original question or task.
- **Template:** A structured format that guides how the model should interpret and respond.

Thoughtful prompt engineering ensures that the model stays focused, avoids hallucinations, and generates responses that are not only coherent but also trustworthy. It essentially acts as a bridge between the retrieved knowledge and the final output, dictating how the information is interpreted and presented.

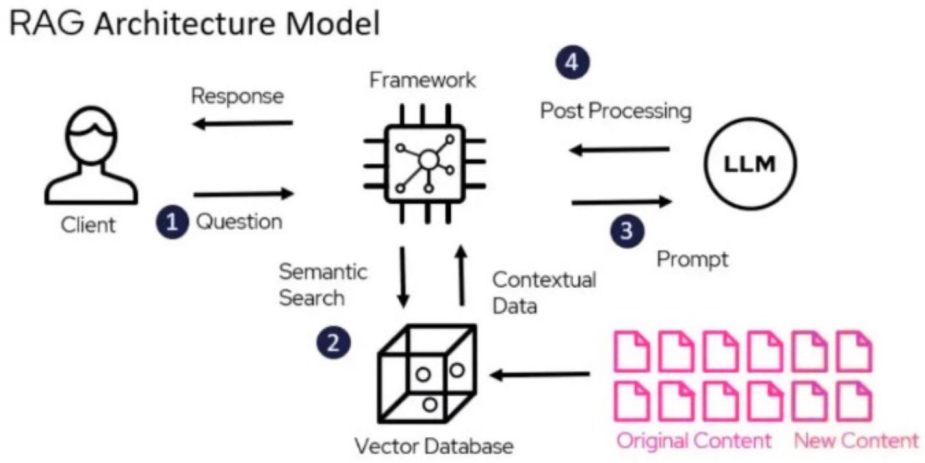


Figure 10: RAG Architecture: Modified from [**12**]

By integrating retrieval into the generation process, RAG systems offer a flexible and efficient solution to build knowledge-based applications. They enable models to answer queries based on live, external sources, without requiring continuous retraining or fine-tuning. This makes RAG especially valuable for use cases that involve large knowledge bases, frequently changing data, or domain-specific expertise. From embedding generation and chunking to vector similarity search and dynamic prompt design, every step in the pipeline contributes to making RAG systems more responsive, explainable, and reliable.

3.7 Practical LLM Considerations

LLMs are powerful tools that combine Natural Language Understanding and Artificial Intelligence. As powerful as these models may be, they are restricted by significant limitations that are important to understand to optimize their functioning.
Input prompts can be exploited to manipulate generative AI systems to produce unwanted or harmful content. This vulnerability is called prompt injection. Users may intentionally craft prompts that bypass content filters, access or delete internal information, reveal hidden system prompts, or generate offensive or inappropriate responses. In real-world applications, prompt injection has been used to make chatbots leak internal configuration instructions. This risk can be mitigated by implementing strong and regularly updated content filters and ensuring they function reliably across a wide range of inputs. In addition, users should be educated about responsible use, and interactions should be actively monitored to detect and respond to misuse. Some developers fine-tune models with adversarial prompts during training to strengthen resistance against prompt injection.

LLMs often struggle with tasks that involve logic, mathematics, or numerical reasoning. They also tend to fall short when dealing with structured domains such as physics or finance, where precision and calculation are critical. This is largely because LLMs are trained on language patterns, not logical rules or mathematical frameworks. However, when combined with computational tools or specialized AI systems, hybrid models can be built that bridge this gap. These models allow LLMs to handle input from natural languages while delegating complex computations.

LLMs are known to hallucinate or fabricate content that sounds plausible but is factually incorrect. This poses a serious risk, especially in domains where accuracy is critical, such as healthcare, law, or finance. While automated fact-checking using machine learning techniques is being explored, it is not always reliable. Human review remains the most accurate method for evaluation and fact verification, even though it can be time consuming and costly. For example, an LLM might confidently state a medical treatment that appears valid but does not have a basis in clinical guidelines.

LLMs can reflect and even amplify biases present in their training data, leading to the propagation of harmful stereotypes or prejudiced language. If the data set itself is unbalanced or biased, the model is likely to mirror those patterns in its outputs. To address this, it is essential to regularly monitor model behaviour, fine-tune with more diverse datasets, and implement strong content filters. Human oversight also plays a key role in identifying and correcting biased responses to ensure fairness and reduce harm.

LLMs are capable of generating text that appears well-researched and credible, often mimicking the tone and structure of academic or factual content. However, they do not have access to real-time information or memory of where specific pieces of training data came from. This can lead to outputs that sound accurate but are entirely fabricated or outdated. To address this limitation, retrieval-augmented generation systems can be used to ground responses in verified, up-to-date sources. Thus, improving both factual accuracy and trustworthiness. Another challenge is interpretability, or understanding why a model made a particular decision or prediction. This lack of transparency makes debugging errors or identifying biases more difficult, especially in high-stakes environments.

Reducing the size of large language models often involves balancing efficiency with performance. There are several common approaches used to make models lighter without fully compromising their capabilities. Simplifying the model architecture can shrink the size of the model, although it may limit how well the model captures complex patterns. Parameter quantization, where values are stored with lower precision, can help save space but may slightly affect accuracy. Knowledge distillation allows a smaller model to learn from a larger one by imitating its output, which improves efficiency, although the smaller model rarely performs at the same level. Pruning involves removing less important connections in the model to streamline computation; retraining is used alongside these methods to improve results. Other useful strategies include optimizing data usage and making the model more compatible with specific hardware. Altogether, the process involves finding the right balance between model size, speed, and capability depending on the use. Ultimately, while LLMs offer tremendous promise, their safe and effective use depends on combining technical safeguards, responsible design choices, and continuous human oversight.

4 Demo Report

4.1 System Setup

4.1.1 Environment Setup

This project was developed on **Windows 11 version 24H2**. The installed Python version was **3.13.2**. The additional tools used were **Visual Studio Code (version 1.101.2)**, **Docker (version 28.2.2, build e6543b4)** and **curl (version 8.13.0)**.

4.1.2 Docker Installation

Docker was installed using **Docker Desktop**. The version installed was **Docker 28.2.2, build e6543b4**. **WSL** was installed specifically to enable Docker to run on Windows. Docker was chosen to simplify ElasticSearch deployment and avoid manual installation and subsequent inconveniences.

4.1.3 ElasticSearch Setup via Docker

ElasticSearch was setup using Docker to simplify deployment and avoid manual installation steps. The official ElasticSearch Docker image used was:

```
docker.elastic.co/elasticsearch/elasticsearch:8.13.4
```

The image was pulled using:

```
docker pull docker.elastic.co/elasticsearch/elasticsearch:8.13.4
```

The ElasticSearch container was then started with the following command:

```
docker run -d --name elasticsearch \
-p 9200:9200 \
-e "discovery.type=single-node" \
docker.elastic.co/elasticsearch/elasticsearch:8.13.4
```

This command runs ElasticSearch as a single node cluster and makes it accessible on *http://localhost:9200*. No additional authentication was configured for local testing.

The installation was completed successfully without significant issues.

4.2 Data Preparation

4.2.1 Dataset Collection

The information in the text files was summarized and adapted from publicly available Wikipedia pages on NLP, NLG, NLU, RAG, Conversational AI, Chat bots, Deep Learning,

Machine Learning, ElasticSearch, and Vector Databases. Ten text files were created, each representing one topic. These topics were selected because of their relevance to the technical context of the chatbot. The data was roughly summarized in addition to removing section headers and references. The text files were saved in .txt format and UTF-8 encoding. Each text file in the data set comprises 280 to 310 words and is 2-3 KB in size. This size was selected to keep the implementation simple and make it convenient to split, index and query the data. However, this data set is limited in depth compared to domain-specific and real-world datasets.

4.2.2 Chunking Text Data

The chunking strategy chosen for this dataset is sentence-based chunking. Sentence-based chunking divides text into complete sentences, which preserves the logical flow of information and splits the document at natural sentence boundaries. The upside of this method is that the meaning of the document is not lost and there is no disruption in the flow of information. The chunks are readable and more coherent, making them easier to process. Cutting the text at logical points preserves the idea of the sentence within each chunk. However, the downside is that chunking based on sentences may be inefficient when sentences are too long or exceed character limits, making it harder to control chunk consistency and potentially altering performance. For this project, the chosen chunk size is one sentence per chunk, with no overlap considered. The NLTK Sentence Tokenizer has been used to split text into sentences. However, some formatting may still be needed in cases where sentence terminators are used in places where a sentence is not actually ending, such as in initials.

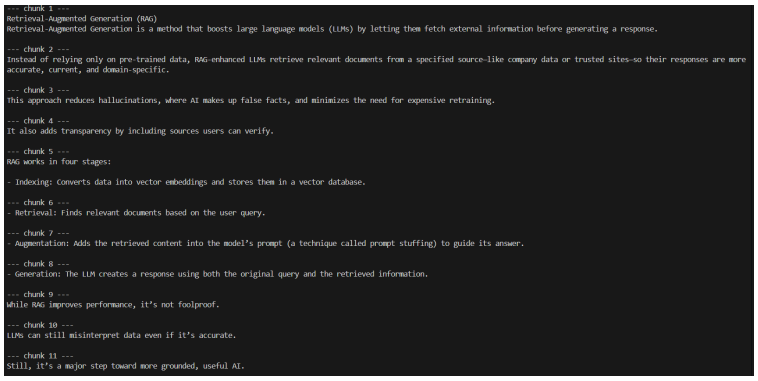


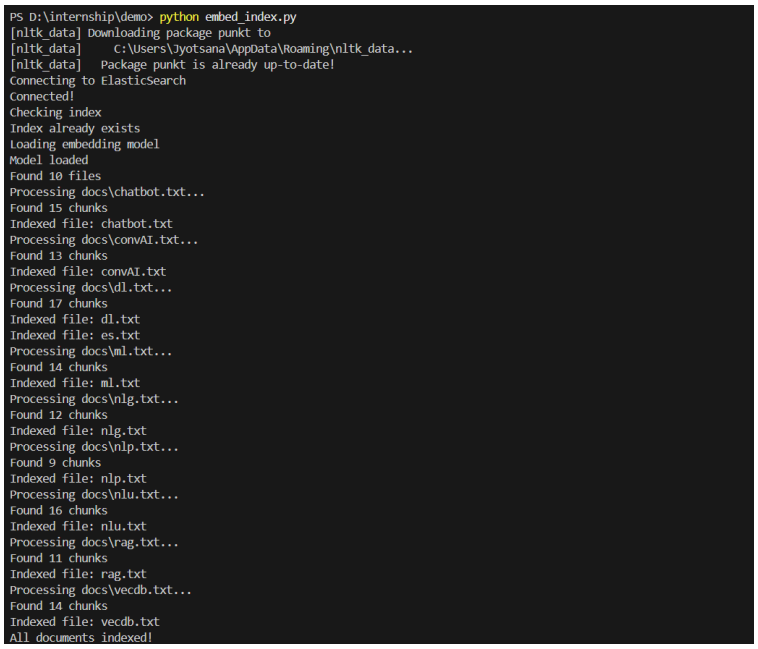
Figure 11: Example of Chunked Text File 'rag.txt'

4.2.3 Indexing into ElasticSearch

An index called **demo_index** was created. The data in each document was chunked using the sentence-chunking technique into chunks comprising one sentence each. These

chunks were converted into 384-dimensional dense vectors using the **All-MiniLM-L6-v2** embedding model from Sentence Transformers. The documents were indexed into four fields:

1. **'filename'**: name of the source text file
2. **'chunk_number'**: position of the chunk in the document
3. **'chunk_text'**: the chunk itself
4. **'embedding'**: the semantic meaning of the chunk, represented as a 384 dimensional embedded vector



```
PS D:\internship\demo> python embed_index.py
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Jyotsana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Connecting to ElasticSearch
Connected!
Checking index
Index already exists
Loading embedding model
Model loaded
Found 10 files
Processing docs\chatbot.txt...
Found 15 chunks
Indexed file: chatbot.txt
Processing docs\convAI.txt...
Found 13 chunks
Indexed file: convAI.txt
Processing docs\dl.txt...
Found 17 chunks
Indexed file: dl.txt
Indexed file: es.txt
Processing docs\ml.txt...
Found 14 chunks
Indexed file: ml.txt
Processing docs\nlg.txt...
Found 12 chunks
Indexed file: nlg.txt
Processing docs\nlp.txt...
Found 9 chunks
Indexed file: nlp.txt
Processing docs\nlu.txt...
Found 16 chunks
Indexed file: nlu.txt
Processing docs\rag.txt...
Found 11 chunks
Indexed file: rag.txt
Processing docs\vecdb.txt...
Found 14 chunks
Indexed file: vecdb.txt
All documents indexed!
```

Figure 12: Output on running indexing code

4.3 Implementation and Experiments

4.3.1 Sparse Search Queries

The ElasticSearch index **demo_index** was created specifically for sparse search. The indexed document fields include **filename**, **chunk_number**, **chunk_size**, and **embedding**. A **match** query was performed on the **chunk_text** field to retrieve the relevant chunks. The sparse retrieval technique used was a keyword-based sparse retrieval through BM25.

```python
from elasticsearch import Elasticsearch

ES_URL = "http://localhost:9200"
INDEX_NAME = "demo_index"

es = Elasticsearch(ES_URL)

while True:
    query_text = input("Enter your search query (or type 'exit' to quit): ").strip()

    if query_text.lower() == "exit":
        print("Demo Terminated.")
        break

    response = es.search(
        index=INDEX_NAME,
        body={
            "query": {
                "match": {
                    "chunk_text": query_text
                }
            }
        }
    )

    hits = response["hits"]["hits"]

    print(f"\nFound {len(hits)} results:\n")

    for hit in hits:
        score = hit["_score"]
        filename = hit["_source"]["filename"]
        chunk_text = hit["_source"]["chunk_text"]

        print(f"Score: {score:.2f}")
        print(f"File: {filename}")
        print(f"Chunk: {chunk_text}")
        print("-" * 50)

    print()
```

Figure 13: Sparse Search Code

```
PS D:\internship\demo> python sparse_demo.py
Enter your search query (or type 'exit' to quit): netural

Found 0 results:


Enter your search query (or type 'exit' to quit): exit
Demo Terminated.
```

Figure 14: Sparse Search Output on User Query: 'netural'

```
PS D:\internship\demo> python sparse_demo.py
Enter your search query (or type 'exit' to quit): understanding

Found 10 results:

Score: 4.24
File: nlg.txt
Chunk: NLG complements natural-language understanding (NLU).
--------------------------------------------------
Score: 4.24
File: nlg.txt
Chunk: NLG complements natural-language understanding (NLU).
--------------------------------------------------
Score: 4.00
File: nlu.txt
Chunk: Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with machine reading comprehension.
--------------------------------------------------
Score: 4.00
File: nlu.txt
Chunk: Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with machine reading comprehension.
--------------------------------------------------
Score: 3.62
File: nlu.txt
Chunk: The interpretation capabilities of a language-understanding system depend on the semantic theory it uses.
--------------------------------------------------
Score: 3.62
File: nlu.txt
Chunk: The interpretation capabilities of a language-understanding system depend on the semantic theory it uses.
--------------------------------------------------
Score: 3.45
File: nlp.txt
Chunk: Major tasks in natural language processing are speech recognition, text classification, natural language understanding, and natural language generation.
--------------------------------------------------
Score: 3.45
File: nlp.txt
Chunk: Major tasks in natural language processing are speech recognition, text classification, natural language understanding, and natural language generation.
--------------------------------------------------
Score: 3.40
File: chatbot.txt
Chunk: Despite creating an illusion of intelligence, such systems lacked genuine understanding and often frustrated users with repetitive, limited replies.
--------------------------------------------------
Score: 3.40
File: chatbot.txt
Chunk: Despite creating an illusion of intelligence, such systems lacked genuine understanding and often frustrated users with repetitive, limited replies.
--------------------------------------------------

Enter your search query (or type 'exit' to quit): exit
Demo Terminated.
```

Figure 15: Sparse Search Output on User Query: 'understanding'

From the above figures, the sparse search results depended on exact word matches. Chunks with multiple repetitions of the query were ranked higher, while those with a looser but still relevant context were ranked lower. The retrieval was mainly focused on literal overlap, ignoring phrasing or intent. File grouping highlighted relevance across documents.

4.3.2 Dense Search and Vector Indexing

The ElasticSearch index **demo_index** was also used for dense retrieval. Each document in the index includes fields such as **chunk_text**, **embedding**, **filename**, **chunk_number**. The all-MiniLM-L6-v2 sentence-transformer model was used to convert the user query into a dense vector representation. A **script_score** query with cosine similarity was then applied to compare this query vector against the pre-stored embeddings within the index. This approach allows for semantic retrieval, meaning that it matches the intent and meaning of the query rather than relying solely on exact keyword matches.

```python
from elasticsearch import Elasticsearch
from sentence_transformers import SentenceTransformer

ES_URL = "http://localhost:9200"
INDEX_NAME = "demo_index"
MODEL_NAME = "all-MiniLM-L6-v2"

es = Elasticsearch(ES_URL)
if not es.ping():
    print("ElasticSearch is not running!")
    exit()

model = SentenceTransformer(MODEL_NAME)

user_query = input("Enter your query: ")

query_vector = model.encode(user_query).tolist()

query_body = {
    "size": 5,
    "query": {
        "script_score": {
            "query": {
                "match_all": {}
            },
            "script": {
                "source": "cosineSimilarity(params.query_vector, 'embedding') + 1.0",  # +1.0 to avoid negative scores
                "params": {
                    "query_vector": query_vector
                }
            }
        }
    }
}

response = es.search(index=INDEX_NAME, body=query_body)

print("\nTop Results:\n")
for hit in response["hits"]["hits"]:
    print(f"[Score: {hit['_score']:.4f}] {hit['_source']['chunk_text'][:200]}...\n")
```

Figure 16: Dense Search Code

```
PS D:\internship\demo> python dense_demo.py
Enter your query: netural

Top Results:

[Score: 1.2021] Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with...

[Score: 1.1962] Components of conversational AI
Conversational AI has principal components that allow it to process, understand and generate response in a natural way....

[Score: 1.1962] Components of conversational AI
Conversational AI has principal components that allow it to process, understand and generate response in a natural way....

[Score: 1.1934] Alternate neural network types, like cerebellar model articulation controllers (CMAC), offer simpler, more convergent training but are less commonly used in deep
learning applications....

[Score: 1.1934] Alternate neural network types, like cerebellar model articulation controllers (CMAC), offer simpler, more convergent training but are less commonly used in deep
learning applications....
```

Figure 17: Dense Search Output on User Query: 'netural'

```
PS D:\internship\demo> python dense_demo.py
Enter your query: what is natural language understanding

Top Results:

[Score: 1.7378] Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with...

[Score: 1.7040] Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with...

[Score: 1.6803] Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with...

[Score: 1.6803] Natural Language Understanding (NLU)
Natural language understanding (NLU) or natural language interpretation (NLI) is a subset of natural language processing in artificial intelligence that deals with...

[Score: 1.6528] The interpretation capabilities of a language-understanding system depend on the semantic theory it uses....
```

Figure 18: Dense Search Output on User Query: 'what is natural language understanding'

From the above figures, the dense search results were based on semantic similarity rather than exact matches. Chunks that aligned closely with the meaning of the query, even without direct keyword overlap, were ranked higher. This allowed for more flexible retrieval, capturing intent, and paraphrased sentences effectively. The use of vector

embeddings helped surface contextually relevant chunks that sparse search might have missed.

4.4 Performance and Observations

The project explored two retrieval methods: sparse and dense. Each used the same ElasticSearch index *(demo_index)* but was optimized for different search styles. Both were evaluated using a custom dataset curated from simplified Wikipedia content, covering relevant NLP and AI topics.

Sparse retrieval, using BM25 and match queries on the *chunk_text* field, prioritized literal keyword overlap. It performed well when the query closely matched the phrasing used in the original documents. For example, when input queries mirrored terminology from the data source, those chunks were successfully retrieved. However, when the query used synonyms or paraphrased phrasing, the relevant chunks were often ranked lower or completely missed. This demonstrated a key limitation: sparse retrieval lacks semantic flexibility.

Dense retrieval, on the other hand, was performed using sentence embeddings from the *all-MiniLm-L6-v2* model. The user query was encoded into a 384-dimensional vector and compared to the *embedding* field of each chunk using cosine similarity. This method prioritized contextual alignment over exact matches. As a result, dense retrieval successfully surfaced semantically relevant chunks even when they did not share keywords with the query.

In general, dense retrieval provided more useful and accurate results, especially for complex or loosely worded queries. Sparse retrieval still held value for quick, direct matches and required fewer computational resources. Both were influenced by the quality of the chunks. Since each chunk was one sentence long, sentence boundaries helped preserve clarity, but some unevenness in sentence length occasionally affected performance.

5 Conclusion

This internship provided a thorough grounding in the theoretical and practical dimensions of conversational AI, with a particular focus on the technologies that underpin modern intelligent systems. Through structured research and hands-on experimentation, the work explored the full arc of chatbot evolution — from rigid, rule-based systems to sophisticated generative AI architectures — and culminated in a working demonstration of information retrieval techniques using ElasticSearch.

The background research established that conversational AI has undergone a fundamental transformation. Early chatbots, constrained by monolithic architectures and pattern-matching logic, gave way to modular systems powered by natural language processing, deep learning, and large language models. The study of intent classification revealed how modern platforms like Dialogflow enable scalable, context-aware interactions, as demonstrated by real-world deployments such as Domino's "Dom" assistant. The comparative analysis of chatbots versus AI agents further highlighted the growing expectation for systems that can reason, act autonomously, and handle complex, multi-turn conversations.

The research into generative AI models — including GANs, VAEs, diffusion models, and transformer-based LLMs — underscored how deeply the field has shifted toward generation over retrieval alone. The transformer architecture, particularly its self-attention mechanism, remains the backbone of state-of-the-art language understanding and generation. However, as explored in the section on practical LLM considerations, these models are not without limitations: hallucination, bias, prompt injection vulnerabilities, and computational cost remain active challenges that require careful mitigation through techniques such as RAG, fine-tuning, and human oversight.

The sections on NLP, NLU, and information retrieval situated these technologies within a broader ecosystem. Understanding the distinction between sparse retrieval — which relies on lexical overlap and algorithms like BM25 — and dense retrieval, which leverages semantic embeddings for contextual matching, was central to the practical component of this internship. The RAG framework emerged as a compelling solution to many of LLMs' core limitations, grounding generation in verified, retrievable knowledge and reducing the risk of fabricated or outdated responses.

The demo project gave concrete form to these ideas. Using a custom dataset of ten text files covering core AI and NLP topics, the project implemented both sparse and dense retrieval pipelines within ElasticSearch. Sentence-based chunking using the NLTK tokenizer was employed to preserve semantic coherence, and the All-MiniLM-L6-v2 model was used to generate 384-dimensional embeddings for dense vector indexing. Experiments confirmed the expected theoretical trade-offs: sparse retrieval excelled at exact keyword matching but failed on paraphrased or misspelled queries, while dense retrieval handled semantic variation effectively, surfacing contextually relevant results even in the absence of direct keyword overlap. The typo query "netural" — which returned zero results under BM25 but yielded accurate results under dense search — served as a clear and practical illustration of this distinction.

Taken together, this internship reinforced that no single technique is universally supe-

rior. The future of intelligent information systems lies in hybrid approaches that combine the precision of sparse retrieval, the semantic depth of dense retrieval, and the generative fluency of LLMs within robust, well-engineered pipelines. As AI continues to advance across healthcare, education, and enterprise applications, the principles explored here — retrieval-augmented generation, responsible deployment, and modular design — will only grow in relevance.

6 References

https://www.ibm.com/think/topics/conversational-ai
https://addepto.com/blog/quick-guide-to-programming-an-ai-chatbot/
https://www.ibm.com/think/topics/chatbots
https://cobusgreyling.medium.com/a-short-history-of-chatbots-42a92a4cf296
https://cloud.google.com/dialogflow/es/docs/intents-overview
https://www.tidio.com/blog/chatbot-intents/
https://www.ibm.com/think/topics/nlp-vs-nlu-vs-nlg
https://www.ibm.com/think/topics/large-language-models
https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/
https://aws.amazon.com/what-is/retrieval-augmented-generation/
https://www.k2view.com/blog/rag-chatbot/RAG-chatbots-and-beyond/
https://hellotars.com/blog/how-to-use-a-chatbot-to-query-database-via-apis/
https://logz.io/learn/complete-guide-elk-stack/latest-on-the-elk-stack/
https://logz.io/blog/elasticsearch-queries/
https://www.f22labs.com/blogs/7-chunking-strategies-in-rag-you-need-to-know/
https://www.linkedin.com/pulse/retrieval-techniques-sparse-dense-hybrid-najeeb-khan-phd–wmtpc/
https://www.geeksforgeeks.org/nlp/what-is-bm25-best-matching-25-algorithm/
https://milvus.io/ai-quick-reference/what-is-the-difference-between-sparse-and-dense-retrieval
https://www.elastic.co/what-is/information-retrievaltypes-of-information-retrieval-models
https://levelup.gitconnected.com/5-practical-limitations-of-llms-49f0f5341fc9
https://dhirajpatra.medium.com/reducing-the-size-of-an-llm-2ef22f01af20
https://www.sprinklr.com/blog/evaluate-llm-for-safety/toc-0
https://towardsdatascience.com/the-complexities-and-challenges-of-integrating-llm-into-applications-913d4461bbe0/
https://www.talkdesk.com/blog/chatbots-talk-ai-agents-think-know-the-difference/
https://medium.com/@dinabavli/rag-basics-basic-implementation-of-retrieval-augmented-generation-rag-e80e0791159d
https://medium.com/@dinabavli/the-evolution-of-nlp-from-embeddings-to-transformer-based-models-83de64244982
https://kanerika.com/blogs/ai-agent-examples/
https://medium.com/@aikho/deep-learning-in-information-retrieval-part-ii-dense-retrieval-1f9fecb47de9
https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview
https://weaviate.io/blog/hybrid-search-explained
https://medium.com/@devalshah1619/mathematical-intuition-behind-reciprocal-rank-fusion-rrf-explained-in-2-mins-002df0cc5e2a
https://www.elastic.co/what-is/information-retrieval

References

[1] State of AI Portal. *AI and Human-Machine Collaboration: A Symbiotic Future.* Available at: https://stateofaiportal.medium.com/ai-and-human-machine-collaboration-a-symbiotic-future-90c8dc720d44 (Accessed: July 24, 2025)

[2] Medium. A Short History of Chatbots. Available at: https://cobusgreyling.medium.com/a-short-history-of-chatbots-42a92a4cf296 (Accessed July 24, 2025)

[3] Digital Architecture Lab Portal. *Modularity in AI: Understanding the Building Blocks of Intelligence.* Available at: https://dalab.xyz/en/blog/modularity-in-ai-understanding-the-building-blocks-of-intelligence/ (Accessed: July 24, 2025)

[4] Google Cloud. *Intents.* Available at: https://cloud.google.com/static/dialogflow/es/docs/images/intent-match-respond-basic.svg (Accessed July 25, 2025)

[5] Google Cloud. *Domino's simplifies ordering pizza using Dialogflow's conversational technology.* Available at: https://cloud.google.com/dialogflow/docs/case-studies/dominos#:~:text=Domino's%20simplifies%20ordering%20pizza%20using%20Dialogflow's%20conversational%20technology&text=Galluch%2C%20Domino's%20program%20leader%20for%20digital%20experience. (Accessed July 25, 2025)

[6] Medium. *Implementing Generative AI: A Pipeline Architecture.* Available at: https://medium.com/@theagipodcast/implementing-generative-ai-a-pipeline-architecture-7321e0a5cec4 (Accessed July 29, 2025)

[7] Cornell University. *Attention is All You Need.* Available at: arXiv:1706.03762 Accessed July 29, 2025

[8] Wikipedia. *Long short-term Memory.* Available at: https://en.wikipedia.org/wiki/Long_short-term_memory Accessed July 29, 2025

[9] Wikipedia. *Bayesian Inference.* Available at: https://en.wikipedia.org/wiki/Bayesian_inference Accessed July 29, 2025

[10] Medium. *Unraveling the Magic: A Deep Dive into Generative AI.* Available at: https://medium.com/@theagipodcast/unraveling-the-magic-a-deep-dive-into-generative-ai-0e34cbf89895 Accessed July 29, 2025

[11] Medium. *Unraveling the Magic: A Deep Dive into Generative AI.* Available at: https://medium.com/@theagipodcast/unraveling-the-magic-a-deep-dive-into-generative-ai-0e34cbf89895 Accessed July 29, 2025

[12] Elastic. *What is generative AI.* Available at: https://www.elastic.co/what-is/generative-ai Accessed July 29, 2025

[13] PubMed Central. *Artificial Intelligence for Medical Diagnostics—Existing and Future AI Technology!* Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9955430/ (Accessed July 25, 2025)

[14] International Journal of Health Sciences. *The Use of Chatbots for triage and Emergency Nursing Support: Review Article.* Available at: 10.5370/ijhs.v5nS1.15150

[15] Academia. *MESTRIMAN: An expert system fo medical triage and clinical management of patients of catastrophes.* Available at: https://www.academia.edu/11731440/MESTRIMAN_An_expert_system_for_medical_triage_and_clinical_management_of_patients_in_catastrophes

[16] PubMed Central. *Artifical intelligence in healthcare: transforming the practice of medicine.* Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8285156/

[17] ResearchGate. *Adaptive Learning through Artificial Intelligence.* Available at: https://www.researchgate.net/publication/372701884_Adaptive_Learning_through_Artificial_Intelligence

[18] IBM. *What is an attention mechanism.* Available at: https://www.ibm.com/think/topics/attention-mechanism Accessed on July 31st, 2025

[19] Analytics Vidhya. An End to End Guide on NLP Pipeline. Available at: https://www.analyticsvidhya.com/blog/2022/06/an-end-to-end-guide-on-nlp-pipeline/ (Accessed July 24, 2025)

[20] Botpress. *What is natural language processing (NLP) in AI?* Available at: https://botpress.com/blog/natural-language-processing-nlp

[21] TutorialsPoint. *NLP: Information Retrieval.* Available at: https://www.tutorialspoint.com/natural_language_processing/images/relevant_output_about_information.jpg Accessed on July 24th, 2024

[22] ADaSci. *Understanding Okapi BM25: A Guide to Modern Information Retrieval.* Available at: https://adasci.org/understanding-okapi-bm25-a-guide-to-modern-information-retrieval/

[23] Glasp. *Understanding the Difference between Sparse and Dense Information Retrieval.* Available at: https://glasp.co/hatch/p5HrypsMODcgZmpmIXWdVk7NiAw1/p/yGVZWMF1H9YkWIfERG0k

[24] LinkedIn. *Retrieval Techniques: Sparse, Dense, and Hybrid Representations.* Available at: https://www.linkedin.com/pulse/retrieval-techniques-sparse-dense-hybrid-najeeb-khan-ph-d--wmtpc

[25] Couchbase. *Hybrid Search: An Overview.* Available at: https://www.couchbase.com/blog/hybrid-search/

[26] Medium. *Hybrid Search - Amalgamation of Sparse and Dense vector representations for Active Content Discovery.* Available at: https://medium.com/@maithri.vm/hybrid-search-amalgamation-of-sparse-and-dense-vector-representations-for-active-con

[27] Tutorials Point. NLP - Information Retrieval. Available at: https://www.tutorialspoint.com/natural\language\processing/natural\language\processing\information\retrieval.htm

[28] ResearchGate. *ES2: A cloud data storage system for supporting both OLTP and OLAP.* Available at: ES2:AclouddatastoragesystemforsupportingbothOLTPandOLAP

[29] Medium. *Designing High Performance RAG Systems.* Available at: https://medium.com/@bijit211987/designing-high-performing-rag-systems-464260b76815